Aurora: An Estate-Oriented Computer Vision MLOps Architecture with Unified Provenance

Jared Mendoza

Aurora

Abstract

Contemporary MLOps tooling fragments the machine learning practitioner's workflow across disconnected systems for experiment tracking, model versioning, dataset governance, and deployment management. This paper presents Aurora, an MLOps workbench architecture that treats these concerns as nodes and edges in a single navigable ontology graph rather than as separate tools. Aurora introduces three contributions:

(1) A three-chain provenance model that unifies training experiment custody, continuous-learning lineage, and W3C PROV-DM formal provenance into a single queryable graph.

(2) A mode-based page architecture over a shared ontology that preserves investigative context across operational modes.

(3) The conceptual framing of MLOps tooling as an *estate* — a habitable environment that compounds operator capability over time, rather than a task-completion interface. We demonstrate the architecture through an end-to-end case study tracking a computer vision model from raw data acquisition through iterative training (including a failed preliminary run and a production restart with heavy augmentation) to deployment with checkpoint-based weight updates.

---

1. Introduction

The operational surface of a machine learning project spans data acquisition, labeling, dataset governance, training orchestration, model versioning, evaluation, deployment, and continuous learning. In practice, practitioners navigate this surface using 10-15 disconnected tools — a web scraper, a labeling interface, a training framework, an experiment tracker, a model registry, a deployment pipeline, and various ad-hoc scripts that bridge the gaps between them. Each tool owns a fragment of the project's history, and no

single system maintains a complete chain of custody from raw data to deployed model to updated weights.

This fragmentation produces three concrete failures. First, **provenance gaps**: when a deployed model exhibits unexpected behavior, tracing backward to the exact dataset version, training configuration, and weight lineage that produced it requires manual reconstruction across multiple systems. Second, **investigation friction**: understanding the relationships between entities (which dataset snapshot trained which model version, which lineage produced which checkpoint, which evaluation range gates which deployment) requires the practitioner to hold a mental model that no tool externalizes. Third, **architectural inversion**: existing tools organize around their own entity types (experiments, runs, artifacts) rather than around the practitioner's investigation flow, forcing the operator to adapt to the tool rather than the tool adapting to the operator.

Aurora addresses these failures through a unified architecture built on three principles. The system maintains a **complete chain of custody** from data intake through deployment and iterative weight updates, backed by cryptographic fingerprinting (SHA-256) at every transition. It externalizes the **relationship graph** between all entity types into a single ontology that is both queryable via API and navigable via a two-plane user interface. And it treats the practitioner's environment as an **estate** — a term we borrow from architecture to denote a space designed for habitation rather than task completion, where sustained occupancy compounds the operator's capability over time.

1.1 Terminology

This section defines terms used throughout the paper, organized from general concepts to system-specific vocabulary. Readers familiar with MLOps infrastructure may skip to Section 1.2.

General Concepts

- **Machine learning (ML):** A method of building software that learns patterns from data rather than being explicitly programmed. A machine learning *model* is the learned artifact — a mathematical function with millions of adjustable parameters (called *weights*) that has been tuned to perform a specific task, such as detecting tigers in photographs.
- **Training:** The process of adjusting a model's weights by exposing it to labeled data (e.g., images with bounding boxes around tigers). Training is computationally expensive and typically runs for many *epochs* — complete passes through the entire dataset.
- **Inference:** Using a trained model to make predictions on new, unseen data. If training is studying for an exam, inference is taking it.
- **Dataset:** The collection of labeled examples used to train a model. In computer vision, this is typically images paired with annotation files describing what appears in each image and where.
- **Checkpoint:** A saved copy of a model's weights at a specific point during training. Checkpoints allow training to be paused, resumed, or branched — similar to save points in a video game.
- **Augmentation:** Artificially expanding a training dataset by applying transformations to existing images — rotation, scaling, color shifts, cropping. This forces the model to learn general features rather than memorizing specific images.

Infrastructure Concepts

- **MLOps (Machine Learning Operations):** The engineering discipline of deploying, monitoring, and maintaining ML models in production. Analogous to DevOps for traditional software, but complicated by the additional dimensions of data versioning, model versioning, and experiment tracking.
- **Provenance:** The documented history of an artifact — where it came from, what produced it, and what transformations were applied. In Aurora, provenance answers the question: "Given this deployed model, what exact data, configuration, and sequence of decisions produced it?"
- **Chain of custody:** A complete, unbroken record of every transition an artifact undergoes from creation to deployment. Borrowed from forensic science, where physical evidence must be tracked through every hand that touches it.
- **Fingerprint (SHA-256 hash):** A cryptographic function that produces a unique fixed-length string from any input data. If even one byte of the input changes, the output changes completely. Aurora uses SHA-256 fingerprints to verify that datasets and model weights have not been altered — the digital equivalent of a tamper-evident seal.
- **Ontology:** A structured representation of entity types and the relationships between them. In Aurora, the ontology defines what kinds of things exist in the system (scenarios, datasets, models, jobs, lineages) and how they connect to each other (a job *produces* a model version, which is *governed by* a dataset snapshot).
- **W3C PROV-DM:** A World Wide Web Consortium standard for representing provenance information. It defines a vocabulary of entities (things), activities (processes that act on things), and agents (actors responsible for activities), connected by relations like *wasGeneratedBy*, *used*, and *wasDerivedFrom*. Aurora mirrors its internal custody records into this standard format for interoperability and formal audit.

Aurora-Specific Terms

- **Scenario:** A named training recipe that binds together a dataset, a model backbone (base architecture), hyperparameters, and augmentation configuration. Scenarios are the primary organizing unit in Aurora — analogous to a "project" in other MLOps tools, but with tighter coupling to the training pipeline.
- **Snapshot:** An immutable record of a model's weights at a specific point in time, identified by the SHA-256 hash of the weight file. Snapshots are linked by parentage: each snapshot knows which prior snapshot it was derived from.
- **Lineage:** An append-only chain of training steps (*drops*) that records the complete history of a model's evolution. If a model is trained on batch A, then updated with batch B, then fine-tuned with batch C, the lineage contains three drops — one for each step — with full custody metadata (data fingerprints, configuration deltas, timestamps) at each link.
- **Drop:** A single step in a lineage. Each drop binds a model snapshot to the data batch and training configuration that produced it. Drops are ordered and immutable — they can be appended but never removed or reordered.
- **Range:** An evaluation configuration that defines test subjects, golden sets (curated benchmark data), and gate criteria (minimum performance thresholds a model must pass before deployment).

- **Catalog:** A hierarchical asset library organized by *sectors* (directory-like groupings). Assets in the catalog carry lineage metadata, connecting physical files to their provenance history.
- **Ecosystem page:** The strategic overview mode in Aurora's interface, rendering the full ontology as an interactive graph topology. Provides orientation: "where does this entity sit in the project, and what depends on it?"
- **Workbench pages:** The tactical operational modes (Train, Collect/Edit, Test Range, etc.), each deeply tailored to a specific workflow phase. All workbench pages share the same underlying ontology and scenario context.

1.2 Tool Categories

We distinguish three categories of software tools by their relationship to the operator:

- A **prosthetic** replaces a capability the operator lacks. It is valuable immediately but creates dependency without growth. Most single-purpose MLOps tools (experiment trackers, model registries) function as prosthetics.
- A **flag** signals identity or affiliation. Dashboard UIs that surface metrics primarily for organizational visibility serve this function.
- An **estate** is a habitable environment that makes the operator more capable over time. It externalizes cognitive load (relationship graphs, provenance chains, evaluation history) in ways that compound with use. The operator who has spent six months in an estate has structurally different capabilities than one who has spent six days, independent of the operator's own skill growth.

Aurora's design target is the third category. The architectural consequences of this choice — using mode-based pages over a shared ontology rather than disconnected entity-centric views, making relationships first-class, preserving investigative context across mode transitions — are detailed in Section 5.

---

2. Related Work

2.1 MLOps Platforms

The dominant MLOps platforms — MLflow [3], Weights & Biases [4], DVC [5], Neptune, and ClearML — each address subsets of the operational surface described in Section 1. MLflow provides experiment tracking and a model registry but no formal dataset custody or continuous-learning lineage. Weights & Biases offers rich experiment visualization and partial artifact versioning but no append-only lineage chains or provenance export. DVC provides dataset versioning via content-addressable storage (functionally similar to Aurora's SHA-256 fingerprinting) but does not maintain training-to-deployment lineage or ontology graphs. Neptune and ClearML offer experiment tracking with varying degrees of metadata management but no W3C PROV-DM compliance or unified entity graphs.

Capability	Aurora	MLflow	W&B	DVC	Neptune
Dataset custody (per-file SHA-256 fingerprint)	Yes	No	No	Yes (DVC hash)	No
Model weight custody (SHA-256 + managed copy)	Yes	Partial (artifact logging)	Partial	Yes	Partial
Continuous-learning lineage (append-only drop chain)	Yes	No	No	No	No
W3C PROV-DM formal provenance export	Yes	No	No	No	No
Unified ontology graph (all entity types)	Yes	No	No	No	No
Mode-based pages over shared ontology	Yes	No	No	No	No
Event sourcing (append-only audit log)	Yes	Partial	Partial	No	Partial
Checkpoint fork and resume (parent_snapshot_id)	Yes	No	Partial	No	No

None of these platforms treat the full custody chain — from raw data fingerprint through continuous-learning drops to formal provenance export — as a single unified concern. Aurora's contribution is not any individual capability in this table, but their convergence in a single ontology graph.

2.2 Note on Convergent Development

Several concepts in this paper bear structural similarity to ideas in the broader HCI and software engineering literature. The estate concept — software as a habitable environment that compounds operator capability — shares characteristics with Gabriel's notion of "habitable software" [6], which emphasizes code that is comfortable to read and modify. Kaptelinin and Nardi's activity theory framework [7] similarly foregrounds the relationship between tools and the human activities they mediate. The two-plane interaction model (ecosystem overview and workbench detail) recapitulates elements of Shneiderman's visual information seeking mantra: "overview first, zoom and filter, then details on demand" [8].

We cite these works for positioning rather than derivation. The concepts in Aurora were developed independently from operational experience with ML tooling — specifically, the daily reality of operating across 15+ distributed tools with no unified environment — rather than from engagement with the HCI or software patterns literature. Where our conclusions converge with prior work, we view this as independent corroboration from different starting points: Gabriel arrives at habitability from software craftsmanship, Kaptelinin from activity theory, Shneiderman from information visualization, and we arrive at it from the concrete failures of fragmented MLOps workflows. Where our work diverges — particularly in the emphasis on cryptographic custody chains and the application of estate thinking to ML-specific ontologies rather than source code — we note the distinction.

This transparency is deliberate. We believe convergent arrival at similar conclusions from different disciplines strengthens rather than weakens the underlying claims, and that honest attribution of intellectual lineage (including the absence of direct influence) is a methodological obligation.

---

3. System Architecture

Aurora is organized as two co-dependent subsystems — a perception layer and an operations layer — that are architecturally inseparable in the same way that circulation and respiration are physiologically inseparable. The perception layer (real-time computer vision inference) produces the artifacts, evaluation signals, and deployment contexts that the operations layer must track; the operations layer (orchestration, state management, ontology construction, provenance) provides the custody, lineage, and governance infrastructure without which the perception layer's outputs are unauditable and unreproducible. Neither subsystem is ancillary to the other. Together they expose both HTTP and WebSocket APIs, and integrate a pipeline layer (training execution, dataset governance, model registration) that serves both.

3.1 Durable State

All operational state is persisted in SQLite databases under a dedicated state directory. This choice prioritizes single-node simplicity, zero-configuration deployment, and transactional consistency for the single-operator use case that Aurora targets. The stores are:

jobs.db — Queued and running training and inference jobs, including full payloads and result references. The job dispatcher uses a ThreadPoolExecutor for concurrent execution.

snapshots.db — Model weight artifacts. Each snapshot records a `weights_sha256` hash of the serialized weights, a `weights_uri` pointing to the managed copy or reference location, an `origin` tag (one of `imported`, `lineage`, or `range_fork`), and a `parent_snapshot_id` establishing the derivation chain. Actual weight files are stored in a co-located `snapshot_weights/` directory.

lineages.db — Continuous-learning chains. A lineage is an append-only sequence of *drops*, where each drop binds a `snapshot_id`, a `data_sha256` fingerprint of the training data batch, optional `source` metadata (including external provenance URIs), and a `training_delta` recording the configuration applied. Drops are indexed and ordered within their parent lineage.

provenance.db — A W3C PROV-DM compliant graph stored as `prov_nodes` (entities, activities, agents identified by `urn:cvops:prov:` URIs) and `prov_edges` (standard PROV relations: `wasGeneratedBy`, `used`, `wasDerivedFrom`, `wasAssociatedWith`, `wasInformedBy`, `hadMember`, `wasAttributedTo`, `specializationOf`, `wasInvalidatedBy`).

ranges.db — Evaluation ranges: test subjects, golden sets, drift detection configuration, and gate criteria for deployment decisions.

catalog.db — A sector-tree organizing assets (images, labels, weight files) into hierarchical collections with full lineage metadata per asset.

3.2 Pipeline Layer

The pipeline layer operates on scenario-driven configuration (`mlops/scenarios/*.yaml`), where each scenario defines a training recipe (backbone, dataset, hyperparameters). Two registry files provide durable metadata:

- `mlops/dataset_registry/<scenario>/*.json` — Immutable dataset fingerprints produced by the governance module at training time. Each file records per-file `content_sha256` hashes, a composite `snapshot_hash`, and quality contract validation results.
- `mlops/model_registry.json` — Published model versions, each carrying a `lineage` block that includes the `dataset_snapshot_id` linking the model to its exact training data fingerprint.

An append-only event log (`mlops/integration/events.jsonl`) records all job lifecycle and catalog events for audit reconstruction.

3.3 Ontology Construction

The `ontology.build_graph()` function joins all durable stores into a single Cytoscape-format graph. Entity types include: `scenario`, `backbone`, `dataset`, `cell`, `dataset_snapshot`, `model_version`, `job`, `model_snapshot`, `lineage`, `range`, `catalog_asset`,

`prov_activity`, `prov_entity`, and `prov_agent`. Edges encode typed relationships (e.g., `produces`, `governed_by`, `derived_from`, `belongs_to`, `has_head`, `evaluates`) plus the full PROV overlay for lineages that have provenance tracking enabled.

Registry-derived lineages (constructed from `model_registry.json` entries) appear as read-only views in the graph. Full PROV persistence applies only to lineages managed through the CV Ops API.

---

4. Provenance Model: Three Chains

Aurora's chain of custody comprises three linked subsystems that converge in the ontology graph and PROV export.

4.1 Chain A: Training Experiment Custody

When a training job executes, the governance module (`pipeline.governance`) performs a dataset snapshot before any training begins. This snapshot:

1. Enumerates every file in the training dataset directory.
2. Computes a `content_sha256` hash for each file.
3. Validates a quality contract (configurable per scenario).
4. Writes an immutable JSON record to `mlops/dataset_registry/<scenario>/`.
5. Produces a composite `snapshot_hash` over all file hashes.

Upon training completion, the model registration step writes a version entry to `model_registry.json` that includes the `dataset_snapshot_id`, binding the model to the exact data that produced it. This binding is the minimum viable provenance claim: given any deployed model version, the system can produce the cryptographic fingerprint of every file that was present in its training set.

4.2 Chain B: Continuous-Learning Lineage

For iterative workflows — where a model is updated with new data batches, fine-tuned, or forked from a prior checkpoint — Aurora provides the lineage abstraction.

A lineage is created via `POST /lineages` with an initial *base* snapshot. This registers the lineage as a PROV collection entity, creates the first drop (drop 0) as a PROV activity, and establishes the `wasGeneratedBy` and `hadMember` relations. Each subsequent `POST /lineages/{id}/drops` appends a new drop that:

- References the new `snapshot_id` (the updated weights).
- Records the `data_sha256` of the new training data batch.

- Optionally includes `prov_used_entities` for external data source URIs.
- Stores the `training_delta` (hyperparameter overrides, configuration changes).
- Timestamps the operation window (`started_at`, `finished_at`).

The snapshot store enforces parentage: each new snapshot's `parent_snapshot_id` points to the prior snapshot in the chain. This creates an immutable derivation history that can be traversed in either direction.

A lineage can also be *forked* via `POST /lineages/{id}/fork`, which creates a new lineage branching from a specific drop. The PROV mirror records this as a `specializationOf` relation, preserving the connection to the parent chain.

4.3 Chain C: W3C PROV-DM Mirror

The provenance store mirrors every lineage lifecycle event into a formal PROV-DM graph using URN-based identifiers (`urn:cvops:prov:snapshot:<id>`, `urn:cvops:prov:lineage:<id>`, `urn:cvops:prov:drop:<lineage_id>:<index>`). The mapping is:

Aurora Operation	PROV-DM Representation
Register snapshot	`entity` of type `cvops:ModelSnapshot`; `wasDerivedFrom` parent snapshot; optional `wasAttributedTo` agent
Create lineage	`entity` collection of type `cvops:LineageCollection`; base drop as `activity`; `wasGeneratedBy`, `hadMember`, `wasAssociatedWith` agent
Add drop	`used` (prior snapshot + declared external entities); `wasGeneratedBy` (new snapshot); `wasDerivedFrom` (new from old); `wasInformedBy` (drop from prior drop); `hadMember` (lineage gains member)
Fork lineage	`specializationOf` (child lineage from parent)

Delete snapshot or lineage	`wasInvalidatedBy` (graph retained; weight row may be purged)

The PROV graph is queryable per-lineage via `GET /lineages/{id}/provenance`, which returns a PROV-JSON closure containing all nodes and edges reachable from that lineage. A bulk backfill endpoint (`POST /provenance/backfill`) can reconstruct the entire PROV graph from `lineages.db` and `snapshots.db`, enabling retroactive provenance construction for lineages that were created before the PROV system was enabled.

4.4 Convergence

The three chains are not isolated systems — they converge in the ontology graph. A single `model_version` node connects to its `dataset_snapshot` (Chain A), its `model_snapshot` within a `lineage` (Chain B), and the corresponding PROV activities and entities (Chain C). This means a single graph traversal from any node can reach the complete custody history of any artifact in the system.

---

5. Interaction Model: Two-Plane Architecture

5. Interaction Model: Mode-Based Pages over Shared Ontology

5.1 Architectural Rationale

The conventional critique of page-based navigation in complex tools — that switching pages drops context and fragments the operator's investigation — is valid when each page owns its own entity scope. In a typical MLOps dashboard, the experiments page shows experiments, the models page shows models, and the datasets page shows datasets; navigating between them resets context, and the relationships between entities exist only in the operator's head.

However, the alternative — a single monolithic view — introduces its own failure: cognitive overload from simultaneous presentation of training logs, metric curves, dataset previews, lineage graphs, and evaluation results. The operator needs different information densities at different phases of work.

Aurora resolves this tension through *mode-based pages over a shared ontology*. The design draws from the same architectural pattern used in professional creative tools such as DaVinci Resolve (which separates Cut, Edit, Fusion, Color, Fairlight, and Deliver into distinct pages while maintaining a shared media pool and timeline). Each page is a deeply tailored environment for a specific operational mode, but

the underlying project state — the scenario selection, the ontology graph, the active entity context — persists across all of them.

5.2 Page Structure

The current implementation provides the following operational modes, accessible via a persistent top navigation rail:

Ecosystem — The strategic overview. Renders the full ontology graph as an interactive topology, with nodes representing scenarios, models, datasets, lineages, jobs, and provenance entities. Provides the "where am I in the project" orientation that anchors all other modes.

Collect/Edit — Data acquisition and curation. Supports web scraping, image import, bounding box annotation, label editing, and quality review. The output of this mode feeds the dataset library that the governance module fingerprints.

Train — Training orchestration and live monitoring. Displays epoch-level progress logs, real-time mAP and loss curves, current metrics (box loss, classification loss, precision, recall, mAP50, mAP50-95), training batch sample previews, and scenario configuration (args.yaml). The scenario sidebar persists, showing all scenarios with their current status (TRAINED, DATASET, TRAINING, EMPTY).

Test Range — Evaluation and gating. Manages golden sets, drift detection, and deployment gates against registered model snapshots.

Queue — Job management. Shows dispatched, running, and completed jobs with their payloads and result references.

Database — Dataset library browser. Navigates the physical dataset directories and their associated metadata.

Data Viz — Visualization workspace for metrics, distributions, and comparative analysis across training runs.

Notes, Cells, 3D, Viewport — Domain-specific workspaces for annotation, cell-level analysis, 3D asset inspection, and live inference preview.

Settings — System configuration, scenario definition, and store management.

5.3 Context Persistence

The critical architectural commitment is not the page layout itself but the *context persistence* across page transitions. The scenario sidebar — visible across all operational modes — serves as the persistent entity anchor. When the operator selects the TIGERTEST scenario on the Train page and then navigates to Test Range or Database, the scenario context carries over. The pages are different workbenches in the same shop, not different shops.

This is reinforced by the ontology graph in the Ecosystem page, which provides the structural context that individual operational pages cannot. A practitioner who has been working in Train mode can switch to Ecosystem to see where the current training job sits in the broader project topology — which dataset snapshot it depends on, which lineage it feeds, which evaluation range will gate its deployment — and then descend back into any operational mode with that structural understanding intact.

The DESCEND/ASCEND interaction from the ecosystem graph (double-click a node to enter the relevant operational mode with that entity's context pre-loaded; ascend to return to the graph with state preserved) extends the mode-based page architecture with a graph-navigated entry point. This means the operator can reach any operational mode either through the top navigation rail (direct mode selection) or through the ontology graph (entity-driven mode selection). The two paths converge on the same page with the same entity context.

5.4 Distinction from Disconnected Dashboards

The difference between Aurora's mode-based pages and a conventional MLOps dashboard with multiple tabs is not visual — both have a navigation bar with labeled sections. The difference is structural:

In a disconnected dashboard, each page queries its own data source and renders its own entity list. The experiments page and the models page may both show the same model, but they arrived at it through different queries, display different metadata, and offer different actions. There is no shared state that connects them.

In Aurora, all pages query the same ontology. The scenario sidebar, the job store, the snapshot store, and the lineage store are shared state that every page can read and write. When the Train page registers a new model version, the Ecosystem page's graph updates to include it. When the Collect/Edit page adds images to a dataset, the Database page reflects the change. The ontology is the single source of truth, and the pages are lenses on it.

---

6. Case Study: Tiger Detection Pipeline

To demonstrate the end-to-end custody model, we trace a complete computer vision workflow: building a tiger detection model from raw web-scraped images through iterative deployment with checkpoint-based weight updates.

6.1 Data Acquisition and Labeling

The pipeline begins with web scraping of tiger images from public sources. Raw images are deposited into the dataset library (`database/tigers/`). A manual labeling phase produces YOLO-format annotation files (one `.txt` per image, containing bounding box coordinates and class indices). Quality review identifies and removes mislabeled, duplicate, or low-resolution images. The labeled dataset is then formalized as a Aurora dataset entity, making it addressable within the ontology.

At this stage, the dataset exists as physical files on disk with no custody metadata. The provenance chain has not yet begun.

6.2 Dataset Formalization and Fingerprinting

When the first training job is submitted, the governance module executes `create_dataset_snapshot()` before training begins. This operation:

- Walks every file in `database/tigers/` (images and labels).
- Computes `content_sha256` for each file.
- Validates the quality contract (minimum image count, label coverage, class balance thresholds).
- Writes an immutable JSON snapshot to `mlops/dataset_registry/tigers/`.
- Produces a composite `snapshot_hash`.

This is the moment custody begins. From this point forward, any modification to the training data — adding images, correcting labels, removing samples — will produce a different `snapshot_hash`, and the system will create a new snapshot rather than mutating the existing one.

6.3 Initial Training: Preliminary Run and Restart

The first training attempt serves as a preliminary validation run — a small dataset (11 images per class) with no augmentation, configured for 250 epochs at 512px input resolution with a batch size of 2. By epoch 62, the scenario reports mAP50 of 0.808 and precision/recall near 0.79, but these numbers are unreliable: the dataset is too small for the metrics to generalize, and the absence of augmentation means the model is memorizing textures rather than learning invariant features. The training loss curve descends and the mAP curve stabilizes, but this is overfitting masquerading as convergence.

This preliminary run is abandoned. In Aurora's custody model, the abandoned run remains in `jobs.db` with its result reference, and its dataset snapshot persists in the registry — nothing is deleted, because the historical record has value even for failed experiments. The practitioner starts a new training job from the same scenario definition with a corrected configuration.

6.4 Training: Production Run with Augmentation

The production training run uses a substantially different configuration:

- **Dataset:** 1,000 training images, 1,000 validation images (approximately two orders of magnitude larger than the preliminary run).
- **Augmentation pipeline:** heavy rotation (angle jitter), grayscale conversion, quality degradation, and scale variation. This synthetically expands the effective training distribution well beyond the 1,000 base images, forcing the model to learn geometric and photometric invariances.
- **Training duration:** 250 epochs at 512px resolution, batch size 2.

At epoch 116 (46% through training), the scenario reports mAP50 progressing through the 36-46% range with the curve still climbing — no plateau evident. Training loss continues its monotonic descent. Unlike the preliminary run's artificially high metrics on memorized data, these numbers reflect genuine learning on a diverse, augmented dataset. The model has 134 epochs of productive training remaining.

This configuration illustrates a custody distinction that the provenance model captures precisely. The `dataset_snapshot` in the registry fingerprints the *base* 1,000 training images — the physical files on disk, each with its `content_sha256`. The augmentation pipeline (rotation, grayscale, quality, scale) is recorded in the scenario's `args.yaml` configuration, which is stored as part of the `training_delta` in the lineage drop. The governance module tracks *what data existed*; the training configuration tracks *what transformations were applied to it*. This separation ensures that two training runs on the same base dataset with different augmentation strategies produce the same `dataset_snapshot_id` but different `training_delta` records — correctly reflecting that the raw data is identical while the training procedure differs.

The training job dispatches through `jobs.db`, and upon completion:

- The model is registered in `model_registry.json` as version 1 of the `tigers` scenario.
- The registry entry includes `lineage.dataset_snapshot_id`, binding this model version to the exact dataset fingerprint from Section 6.2.
- The job result is recorded in `jobs.db` with a `result_ref` pointing to the model artifacts.
- An event is appended to `events.jsonl` recording the job completion.

Chain A is now complete for this experiment. Given the model version identifier, the system can produce the exact list of files (with their SHA-256 hashes) that were present during training, and the exact augmentation configuration that was applied to them.

6.5 Deployment as Initial Checkpoint

The trained weights are registered as a model snapshot via `POST /snapshots`:

- `weights_sha256` is computed over the serialized weight file.
- `origin` is set to `imported` (first entry in the system).
- `parent_snapshot_id` is null (no prior snapshot).
- The weight file is copied to `snapshot_weights/` for managed storage.

A lineage is then created via `POST /lineages`, designating this snapshot as the base. Drop 0 is automatically created, establishing the starting point of the continuous-learning chain.

6.6 Iterative Update: New Data Batch

A second batch of tiger images is acquired — perhaps from a different source, or correcting edge cases identified during initial deployment (nighttime images, unusual poses, partially occluded subjects). After labeling and quality review, the update cycle proceeds:

1. The new data batch is fingerprinted (`data_sha256`).
2. Training resumes from the deployed checkpoint with the new data.
3. A new weight file is produced.
4. The new weights are registered as a snapshot with `parent_snapshot_id` pointing to the initial snapshot and `origin` set to `lineage`.
5. A drop is appended to the lineage via `POST /lineages/{id}/drops`, binding the new snapshot to the new data fingerprint and recording the training delta.

Chain B now has two links. The lineage records that snapshot 1 was derived from snapshot 0 using data batch with hash X and training configuration delta Y.

6.7 Checkpoint Fork: Alternative Training Strategy

After evaluating the updated model, the practitioner decides to explore an alternative approach — perhaps a different backbone or a modified augmentation strategy. Rather than starting from scratch, they fork from the initial checkpoint:

- `POST /lineages/{id}/fork` at drop 0, creating a new lineage branching from the original base snapshot.
- Training proceeds with the alternative configuration, producing a new snapshot in the forked lineage.
- Both lineages coexist in the ontology graph, sharing a common ancestor but diverging in their training strategies.

This fork-and-explore pattern is the operational equivalent of branching in version control, but applied to the weight-data-configuration triple rather than source code alone.

6.8 Provenance Export

At any point, the practitioner can request `GET /lineages/{id}/provenance` to obtain the PROV-JSON closure for a lineage. This document contains:

- Every model snapshot as a PROV entity with its `weights_sha256`.
- Every drop as a PROV activity with timestamps and data fingerprints.
- Every derivation relation (`wasDerivedFrom`, `wasGeneratedBy`, `used`).
- The agent attribution (`wasAssociatedWith`) for the cvOps service.
- For forked lineages, the `specializationOf` relation to the parent.

This export is suitable for external audit, regulatory compliance, or integration with other PROV-compatible systems.

6.9 Ontology View

The complete tiger detection project — dataset snapshots, model versions, training jobs, lineage chains, evaluation ranges, catalog assets — appears as a connected subgraph in the ontology. A single graph traversal from the deployed model snapshot reaches every artifact and decision that produced it.

[TODO: When the ecosystem plane is fully wired for this pipeline, add a description of the graph topology as rendered in the UI — node types, edge types, cluster structure, and the visual encoding of provenance depth.]

---

7. Discussion

7.1 Limitations

Dual lineage systems. Aurora currently maintains two lineage representations: the formal continuous-learning lineages in `lineages.db` (with full PROV persistence) and the lighter-weight registry metadata on `model_version` entries in `model_registry.json` (which carry only a `dataset_snapshot_id`). Unifying these into a single lineage abstraction is planned but not yet implemented. In practice, this means that models trained through the standard pipeline have Chain A custody, while only models explicitly managed through the lineage API have Chain B and Chain C custody.

Single-operator scope. The SQLite-backed architecture is designed for single-operator use. Multi-user concurrent access would require migrating to a server-backed database (PostgreSQL, for example) and introducing access control. The current system makes no concurrency guarantees beyond SQLite's default serialized writes.

External pipeline integration. The Insight perception layer and alternative CV stacks (cv.py, Base_Cv_program) sit outside the CV Ops provenance pipeline unless the operator explicitly registers snapshots and lineages through the API. Automatic provenance capture for these external pipelines is not yet implemented.

Evaluation integration. The ranges store provides the schema for evaluation custody (test subjects, golden sets, drift gates), but the integration between range evaluation results and deployment decisions is not yet fully automated. The practitioner must currently interpret range results and manually gate deployments.

7.2 Design Decisions

SQLite over a dedicated database server. The single-file-per-store approach eliminates deployment dependencies and makes the entire state directory trivially portable (copy the directory, move the project).

For the single-operator use case, SQLite's write throughput (~50,000 inserts/second) far exceeds the event rate of any realistic ML workflow.

Append-only semantics for lineages. Drops cannot be removed or reordered within a lineage. This mirrors the physical reality of training — you cannot un-train a model — and simplifies the provenance model. If an experiment needs to be abandoned, the lineage is closed (state set to terminal) rather than deleted, preserving the historical record.

SHA-256 for fingerprinting. Collision resistance is not the primary concern (the fingerprints are not used for security). The choice of SHA-256 over faster hashes (xxHash, BLAKE3) reflects the desire for compatibility with external provenance systems and regulatory frameworks that specify SHA-2 family hashes.

PROV-DM as the formal provenance model. W3C PROV-DM was chosen for interoperability rather than expressiveness. The model is well-specified, has multiple serialization formats (PROV-JSON, PROV-N, PROV-O), and is recognized by standards bodies. The overhead of maintaining the PROV mirror alongside the native lineage representation is modest (typically <5% of lineage operation latency) and is justified by the ability to export standards-compliant provenance documents.

---

8. Conclusion

Aurora demonstrates that the fragmented MLOps tool landscape is not an inevitable consequence of the domain's complexity, but a design choice that can be revisited. By treating provenance as a first-class architectural concern — maintained across three linked chains and surfaced through a unified ontology graph — the system provides complete chain of custody from raw data through iterative deployment without requiring the practitioner to manually reconstruct relationships across disconnected tools.

The estate framing offers a design criterion that existing MLOps tools do not explicitly target: the system should make the operator more capable over time, not merely more efficient at individual tasks. Whether this criterion produces measurably different outcomes is an empirical question that requires longitudinal evaluation beyond the scope of this paper. What we can demonstrate is that the architectural commitments it implies — unified provenance, relationship-first ontology, context-preserving view transitions — are implementable at reasonable engineering cost and produce a qualitatively different interaction with the ML development lifecycle.

Future Work

Four extensions are planned that expand Aurora's operational surface beyond its current object detection focus.

Continuous learning automation. The lineage and drop abstractions described in Section 4.2 currently require explicit API calls to append training steps. The next phase integrates continuous learning as an automated pipeline mode: when new labeled data arrives in a monitored catalog sector, the system

automatically creates a drop, trains from the current head snapshot, evaluates against the active range, and either promotes or rejects the updated weights — all with full provenance recording. The operator reviews and gates; the system handles the mechanical custody chain. This closes the gap between the manual lineage workflow demonstrated in the case study and the fully autonomous update cycle that production deployments require.

Instance segmentation. Aurora's current training pipeline targets object detection (bounding box regression and classification). Extending to instance segmentation — per-pixel mask prediction — introduces new custody requirements. Segmentation masks are substantially larger than bounding box labels, which affects dataset fingerprinting performance (SHA-256 over dense mask files vs. lightweight annotation text files). The governance module's quality contract must expand to validate mask coverage, boundary precision, and class-pixel distribution. The training pipeline must support mask-aware architectures (YOLO-Seg, Mask R-CNN) while maintaining the same scenario-driven configuration and provenance chain. The ontology gains a new entity type (segmentation mask) with edges connecting masks to their source images, annotations, and the model versions trained on them.

3D rendering and spatial reconstruction. A planned pipeline extension integrates RGB-to-3D reconstruction: video capture produces depth maps (via monocular depth estimation), which are fused into volumetric representations (TSDF) and extracted as triangle meshes. This introduces a fundamentally different artifact type into the ontology — 3D meshes with known camera poses, depth confidence maps, and spatial registration metadata. The provenance model must track the chain from source video frames through depth estimation model (itself a versioned artifact in the snapshot store) through volumetric fusion parameters to final mesh. The custody question becomes: given this 3D reconstruction, what exact video frames, depth model version, and fusion configuration produced it? The existing three-chain provenance architecture extends naturally to this domain, with video frames replacing images, depth models replacing detection models, and mesh quality metrics replacing mAP.

Tabular dataset integration. Aurora currently assumes image-based datasets (YOLO-format directories of images and label files). Expanding to tabular data — CSV, TSV, structured databases — requires rethinking several assumptions. Dataset fingerprinting must handle row-level and column-level hashing (not just file-level), schema validation replaces image quality contracts, and the training pipeline must support non-vision architectures (gradient-boosted trees, tabular transformers, classical ML models). The augmentation concept translates differently for tabular data: synthetic minority oversampling, feature noise injection, and missing-value imputation replace geometric and photometric image transforms. The scenario abstraction generalizes cleanly (a scenario is still a named recipe binding data to model to configuration), but the governance module and the training dispatcher require modality-aware extensions. This expansion transforms Aurora from a computer vision operations platform into a general-purpose ML operations estate.

References

Note on citation practice: References [6], [7], and [8] are cited for positioning and convergent validation, not derivation. The concepts they describe were arrived at independently in Aurora's development; the citations establish landscape awareness and identify where independent lines of reasoning converge. See Section 2.2 for a full discussion of this methodological choice.

1. W3C. PROV-DM: The PROV Data Model. W3C Recommendation, April 2013.
2. Moreau, L., Groth, P. *Provenance: An Introduction to PROV.* Synthesis Lectures on the Semantic Web. Morgan & Claypool, 2013.
3. Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S.A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., Zumar, C. Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39-45, 2018.
4. Biewald, L. Experiment Tracking with Weights and Biases. Software available from wandb.com, 2020.
5. Iterative. DVC: Data Version Control. Software available from dvc.org, 2020.
6. Gabriel, R.P. *Patterns of Software: Tales from the Software Community.* Oxford University Press, 1996. [Cited for convergent validation of the "habitable software" concept; see Section 2.2.]
7. Kaptelinin, V., Nardi, B.A. *Acting with Technology: Activity Theory and Interaction Design.* MIT Press, 2006. [Cited for convergent validation of tool-activity mediation framing; see Section 2.2.]
8. Shneiderman, B. The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations. *Proceedings of IEEE Symposium on Visual Languages*, pp. 336-343, 1996. [Cited for convergent validation of the overview-zoom-detail interaction pattern; see Section 2.2.]
1.